Trabajo Práctico 2

Compresión de imágenes sin pérdida mediante codificación de Huffman

Grupo 04
Tomás Cordon, Emanuel Lincic

Inferencia y Estimación — Primavera 2026

Resumen

Se comprimieron cinco imágenes en escala de grises con el código de Huffman, directamente píxel a píxel y sobre los residuos entre píxeles vecinos. Se midieron la entropía, la longitud media del código y la información mutua entre vecinos. Los residuos mejoraron fuertemente la compresión en cuatro de las cinco imágenes (entre 1,6× y 2,8×, frente a 1,05×–1,27× píxel a píxel) pero empeoraron en la imagen con ruido granular (`img_02`), donde la información mutua entre vecinos es casi nula.

1. Introducción

En este trabajo se estudia la compresión sin pérdida de imágenes en escala de grises con el código de Huffman [1], comparando dos representaciones: (i) cada intensidad de píxel como símbolo de la fuente, y (ii) la codificación diferencial mediante residuos entre píxeles vecinos [3]. La entropía de la fuente es el límite inferior para la longitud media de cualquier código sin pérdida [2]. El objetivo es analizar cómo la dependencia entre píxeles vecinos, medida con la información mutua, afecta la eficiencia de la compresión.

2. Metodología

2.1. Imágenes utilizadas

Se trabajó con cinco imágenes: `img_01` (camellos en el desierto, 568×400), `img_02` (textura de arena, 612×408) e `img_04` (partitura, 322×224), provistas por la cátedra, y dos imágenes agregadas por el grupo: `img_03` (festejo del gol de Paredes al Vasco da Gama, 1256×620) e `img_05` (festejo icónico de Messi, 1605×986). Las dos últimas son bastante más grandes que las otras. Todas están en formato JPEG, es decir, ya fueron comprimidas con pérdida: la compresión de este trabajo es sin pérdida respecto de la imagen decodificada. Las imágenes son RGB y se convirtieron a escala de grises de 8 bits (intensidades 0–255) con la conversión de luminancia de la librería Pillow ($L = 0{,}299R + 0{,}587G + 0{,}114B$). Se muestran en la Figura 1 (ancho×alto).



Figura 1: Imágenes utilizadas, en escala de grises.

Los archivos exactos de las imágenes agregadas por el grupo están disponibles en `https://drive.google.com/drive/folders/1oRCDxEhOM-QktOgnfQJCaMPLWKGkGsiX?usp=sharing`.

2.2. Estimación de probabilidades y entropía

La masa de probabilidad se estima empíricamente a partir de las frecuencias relativas de cada valor en la imagen (el estimador de máxima verosimilitud de una distribución discreta). La entropía es

$$H(X) = -\sum_{x} p(x) \log_2 p(x). \tag{1}$$

2.3. Codificación de Huffman

Se utilizó la implementación provista por la cátedra para construir el código y decodificar. Al codificar píxel a píxel se trata a cada intensidad como un símbolo de una fuente de símbolos independientes, con masa de probabilidad $p(x)$ estimada a partir de la propia imagen. La longitud media teórica es $L = \sum_x p(x)\, l(x)$, que cumple $H(X) \leq L < H(X) + 1$. Los bits promedio reales se calculan como la longitud de la cadena codificada dividida por la cantidad de símbolos.

2.4. Residuos

Recorriendo cada fila de izquierda a derecha, el residuo es

$$y_{i,j} = x_{i,j} - x_{i,j-1}, \qquad j = 2, \ldots, N, \tag{2}$$

guardando el primer píxel de cada fila como condición inicial (8 bits por fila, incluidos en el costo total). No se calcula residuo entre el último píxel de una fila y el primero de la siguiente. La reconstrucción es $x_{i,j} = x_{i,j-1} + y_{i,j}$. Se verificó que la reconstrucción es exacta (sin pérdida) en todas las imágenes.

2.5. Información mutua entre vecinos

Para los pares $(X, X') = (x_{i,j}, x_{i,j-1})$ (un píxel y su vecino anterior), usados en los residuos, se estima

$$I(X; X') = H(X) + H(X') - H(X, X') = H(X) - H(X \mid X'), \tag{3}$$

con $H(X, X')$ calculada a partir del histograma conjunto y $H(X \mid X') = H(X, X') - H(X')$ la incertidumbre que queda sobre X al conocer X' (entropía condicional). Cabe aclarar que este estimador, calculado con el histograma conjunto de una cantidad finita de píxeles, está sesgado hacia arriba (resulta no negativo aun si X y X' fueran independientes), y el sesgo es mayor en imágenes con menos píxeles. Por eso valores pequeños de $I(X; X')$ deben interpretarse como cercanos a cero.

3. Resultados

3.1. Ejercicio 1: codificación píxel a píxel

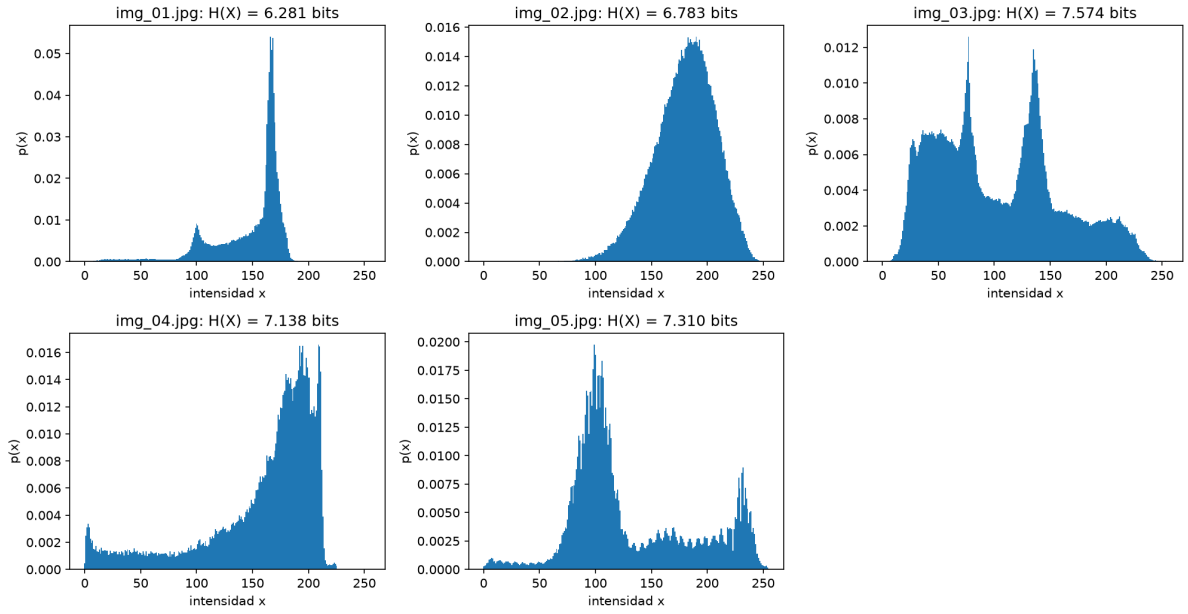


Figura 2: Masa de probabilidad $p(x)$ de cada imagen, con su entropía $H(X)$.

Imagen	$H(X)$ [bits]	L [bits]	Bits prom. reales	Razón de compresión
img_01	6,281	6,310	6,310	1,268
img_02	6,783	6,809	6,809	1,175
img_03	7,574	7,600	7,600	1,053
img_04	7,138	7,165	7,165	1,117
img_05	7,310	7,338	7,338	1,090

Cuadro 1: Resultados del Ejercicio 1. Razón de compresión respecto de 8 bits/píxel.

En las cinco imágenes se cumple $H(X) \leq L < H(X) + 1$, con L solo entre 0,026 y 0,029 bits por encima de $H(X)$. Los bits promedio reales coinciden con L porque el código se construye con la misma PMF con que se mide (no se cuenta el costo de almacenar la tabla del código). Las razones de compresión son modestas (1,05 a 1,27), ya que el código solo aprovecha la forma del histograma.

3.2. Ejercicio 2: codificación de residuos

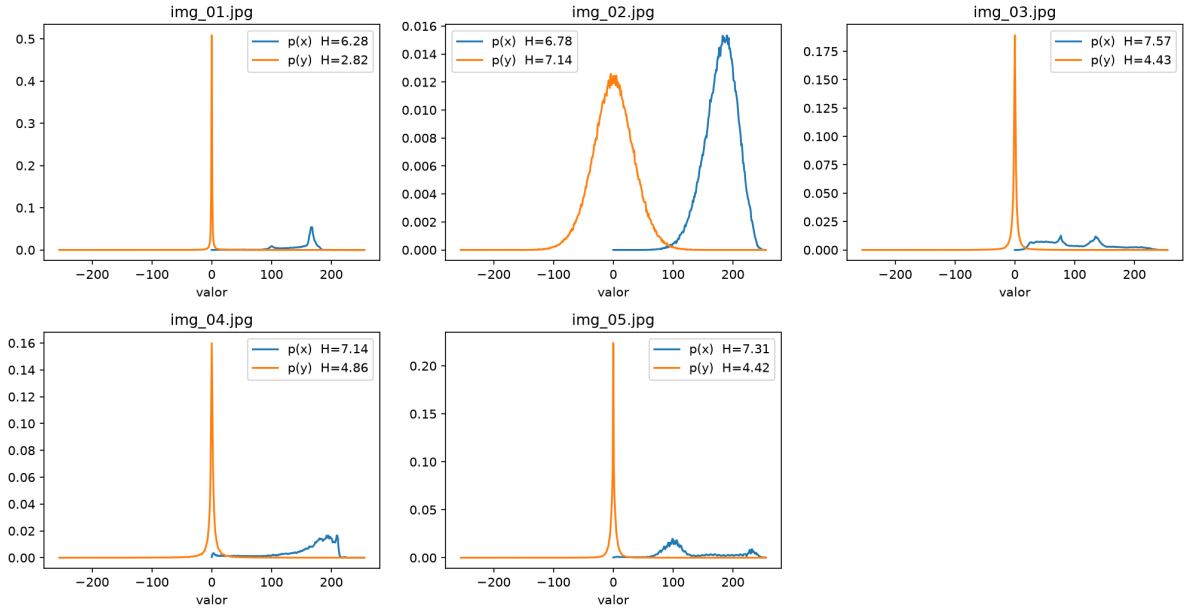


Figura 3: $p(x)$ y $p(y)$ superpuestas para cada imagen, con sus entropías. Con una escala común, $p(x)$ se ve baja porque $p(y)$ concentra casi toda su masa en cero; la forma de $p(x)$ puede verse en la Figura 2.

Imagen	$H(X)$	$H(Y)$	L_Y	Bits prom. reales (res.)	$I(X;X')$
img_01	6,281	2,824	2,838	2,847	3,725
img_02	6,783	7,138	7,179	7,180	0,117
img_03	7,574	4,431	4,462	4,465	3,297
img_04	7,138	4,863	4,889	4,898	2,782
img_05	7,310	4,415	4,452	4,454	3,075

Cuadro 2: Resultados del Ejercicio 2 (en bits). La columna de bits promedio reales incluye el costo de los píxeles iniciales (8 bits por fila).

Los bits promedio de los residuos solos coinciden con L_Y (2,838; 7,179; 4,462; 4,889 y 4,452); al incluir los píxeles iniciales, el costo por píxel cambia en menos de 0,01 bit respecto de L_Y. Se verificó que decodificar y sumar los residuos reconstruye exactamente cada imagen.

3.3. Ejercicio 4: compresor y descompresor `.tp2`

Esta parte es opcional y no se desarrolla en el material de la cátedra: el diseño del formato `.tp2` y las herramientas para implementarlo (lectura de parámetros por terminal con `argparse`, empaquetado de bits en bytes con `numpy` y escritura binaria con `struct`) fueron investigados por el grupo por cuenta propia, a partir de la documentación de Python y de NumPy [4]. Se implementaron dos programas de línea de comandos: `comprimir_tp2.py imagen.jpg` genera `imagen.tp2`, y `descomprimir_tp2.py imagen.tp2` reconstruye y muestra la imagen (con `-o` la guarda además en PNG). El formato binario (módulo `formato_tp2.py`) guarda: un identificador, el alto y ancho, la tabla de Huffman de los residuos (símbolo, largo y código empaquetado), el primer píxel de cada fila (1 byte por fila) y los bits de los residuos empaquetados en bytes. Se verificó que la imagen descomprimida es idéntica a la imagen en escala de grises original, también

en casos borde (imagen constante, de una fila o una columna, y de un solo píxel; ver la celda de pruebas del notebook).

Imagen	Tamaño `.tp2` [bytes]	Bits/píxel (archivo)	Razón vs. 8 bits
img_01	82 197	2,894	2,76
img_02	225 871	7,237	1,11
img_03	436 578	4,485	1,78
img_04	45 993	5,101	1,57
img_05	883 294	4,465	1,79

Cuadro 3: Tamaño real de los archivos `.tp2`. Incluyen la tabla de Huffman, el encabezado y los píxeles iniciales.

Los bits por píxel del archivo son algo mayores que los del Ejercicio 2 (2,85; 7,18; 4,47; 4,90; 4,45) por el costo de almacenar la tabla de Huffman y el encabezado, que antes no se contaba. Esa diferencia es de 0,05 bit por píxel en `img_01` y `img_02`, de 0,2 en la imagen más chica (`img_04`) y de 0,02 o menos en las dos más grandes (`img_03` e `img_05`): el costo fijo de la tabla pesa menos cuanto mayor es la imagen.

4. Análisis

4.1. Ejercicio 3(a): diferencias de entropía entre imágenes

La entropía mide cuán dispersa es $p(x)$ (Figura 2). `img_01` tiene la menor (6,28): grandes zonas de arena casi uniformes producen un pico marcado cerca de $x = 167$. `img_02` (6,78) tiene una campana ancha, de textura homogénea pero con grano. `img_04` (7,14) combina fondo claro, notas y sombras oscuras, y reparte la probabilidad en casi todo el rango. `img_05` (7,31) tiene un pico en tonos medios (el fondo, cerca de $x = 100$) y otro en tonos claros (la camiseta), con mucha textura en el medio. `img_03` tiene la mayor entropía (7,57): una escena compleja, con público y fondo oscuros, tonos medios y camisetas claras, cuyo histograma es multimodal y cubre casi todo el rango de intensidades. En resumen, la entropía depende de cuántos niveles de gris se usan y de cuán uniformemente. Notar que $H(X)$ depende solo del histograma: si se reordenaran los píxeles al azar, no cambiaría, aunque se destruyera toda la estructura espacial.

4.2. Ejercicio 3(b): significado de la información mutua

$I(X; X')$ cuantifica cuánta incertidumbre sobre un píxel se elimina al conocer su vecino, es decir, cuán predecible es. Es alta en `img_01` (3,73 bits), con zonas suaves; algo menor en las fotografías `img_03` (3,30) e `img_05` (3,08), que tienen grandes regiones suaves pero también bordes y texturas; menor en `img_04` (2,78), con bordes marcados; y casi nula en `img_02` (0,12), donde el grano es ruido prácticamente independiente entre vecinos. Se relaciona con el beneficio de los residuos: $H(X) - H(Y)$ vale 3,46; 3,14; 2,90; 2,27 y $-0{,}36$ respectivamente, en el mismo orden que $I(X; X')$. Además, siempre $I(X; X') \geq H(X) - H(Y)$: como Y y X se determinan mutuamente dado X', vale $H(X \mid X') = H(Y \mid X') \leq H(Y)$, y entonces $I(X; X') = H(X) - H(X \mid X') \geq H(X) - H(Y)$. Es decir, $I(X; X')$ es una cota superior de la reducción de entropía que puede lograrse usando al vecino anterior, y los residuos simples (que restan el vecino pero no condicionan a él) capturan solo una parte.

4.3. Ejercicio 3(c): píxel a píxel vs. residuos

Los residuos reducen los bits por píxel de 6,31 a 2,85 en `img_01` (2,81× de compresión frente a 1,27×), de 7,60 a 4,47 en `img_03` (1,79× frente a 1,05×), de 7,34 a 4,45 en `img_05` (1,80×

frente a 1,09×) y de 7,17 a 4,90 en `img_04` (1,63× frente a 1,12×). En estas cuatro imágenes, $p(y)$ es un pico muy concentrado en cero, porque los vecinos son similares, y esa estructura espacial es la redundancia que Huffman píxel a píxel no puede ver. Las dos fotografías (`img_03` e `img_05`) son las de mayor entropía píxel a píxel, o sea las que peor comprimen con ese método (1,05× y 1,09×), y sin embargo con residuos ganan cerca de 3 bits por píxel: la entropía de $p(x)$ y la correlación entre vecinos son propiedades distintas. En `img_02` ocurre lo contrario: los residuos empeoran (7,18 contra 6,81 bits). Si los valores de píxeles vecinos fueran independientes y de varianza σ^2, la diferencia de dos píxeles tendría varianza $2\sigma^2$, así que $p(y)$ es más ancha que $p(x)$ (Figura 3) y su entropía aumenta. Como referencia, para distribuciones gaussianas ese aumento sería de $\frac{1}{2}\log_2 2 = 0{,}5$ bit; aquí se midieron 0,36 bits, sin haber verificado que las distribuciones sean gaussianas, y con $I(X;X') = 0{,}12$, es decir, vecinos casi independientes. De hecho, la desigualdad de 3(b) da $H(Y) \geq H(X) - I(X;X')$, así que con $I(X;X') \approx 0$ los residuos no pueden tener menos entropía que los píxeles. Esta imagen casi no tiene redundancia entre vecinos que explotar. Incluir los píxeles iniciales cambia el costo en menos de 0,01 bit por píxel.

5. Conclusiones

Huffman es óptimo entre los códigos de prefijo que codifican símbolo a símbolo con una masa de probabilidad dada: la longitud media quedó a menos de 0,03 bits de la entropía en todos los casos. Pero solo explota la forma de la PMF, no la dependencia entre píxeles. La codificación por residuos modela esa dependencia y baja la entropía cuando hay correlación espacial: en las cuatro imágenes con $I(X;X')$ alta, los bits por píxel bajaron de 6,3–7,6 a 2,8–4,9 (razones de 1,6× a 2,8×). En `img_02`, con ruido granular y $I(X;X') \approx 0$, los residuos empeoraron (7,18 contra 6,81). La información mutua entre vecinos es entonces un buen predictor de cuánto ayuda la codificación diferencial, y la entropía de $p(x)$ por sí sola no lo es (las fotografías de mayor entropía fueron de las que más ganaron). En los Ejercicios 1 y 2 no se contó el costo de almacenar la tabla de Huffman; el Ejercicio 4 sí lo incluye, y resultó despreciable en las imágenes grandes ($\leq 0{,}02$ bit por píxel) pero no en las chicas (0,2 bit en `img_04`). Como limitaciones, se usó un único esquema de residuos (vecino izquierdo, recorrido por filas), las imágenes de partida ya estaban comprimidas con pérdida (JPEG) y la información mutua se estimó con un estimador sesgado hacia arriba, por lo que los valores pequeños deben leerse como aproximados.

Referencias

[1] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[2] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.

[3] T. Stathaki, *Methods and Standards for Lossless Compression*, Digital Image Processing, Part 4: Image Compression, Imperial College.

[4] Documentación oficial de Python (módulos `argparse` y `struct`) y de NumPy (`packbits`, `unpackbits`), `https://docs.python.org/3/` y `https://numpy.org/doc/`.